

Front-End Developer - Home Assignment

Video Browser Application

Thank you for taking the time to work on this assignment! This exercise is designed to assess your front-end development skills by building a **Video Browser** application that allows users to browse and filter videos by various parameters.

Overview

You will build a single-page application that displays a collection of music videos with filtering capabilities. The application should be responsive, user-friendly, and follow modern React development best practices.

Estimated Time: 4-6 hours

Video Browser



All In
Pants Velour
2014



Veni paca to
El Koala
2014



I Should Have Known It
Tom Petty and the Heartbreakers
2010



Grove St. Party
Waka Flocka Flame
2011



Single Ladies (Put a Ring on It)
Beyoncé
2008



Something Like Olivia
John Mayer
2013



Army of Two
Olly Murs
2013



Wrapped Up
Olly Murs ft. Travie McCoy
2014



Personal Jesus
Fielfraz
1991

Requirements

1. Header Panel

The header should contain the following elements:

a. Application Title

- Display "Video Browser" as the main header

b. Search Video Filter

- A text input field that filters videos by **artist name** OR **title**
- Real-time filtering as the user types (no submit button required)
- No autocomplete functionality needed

c. Search by Year Filter

- Dropdown menu showing available years from the dataset
- Single selection only (one year at a time)
- Should include an option to clear/show all years

d. Search by Genre Filter

- Dropdown menu showing available genres from the dataset
- Multiple selection allowed
- Same genre cannot be selected more than once
- Should include an option to clear selections

Filter Behavior

- **All filters can be combined** - for example: search for "John Mayer", year "2013", and "Rock" genre simultaneously
- Filters should work together using AND logic
- **Bonus (Optional):** Update the year/genre dropdown options dynamically based on the current text search results
- **Empty State:** Display a friendly message "No videos were found" when no results match the current filters

2. Video Card List

Layout Requirements

- Responsive grid layout
- **Minimum:** 1 video per row (mobile)
- **Maximum:** 3 videos per row (desktop)
- Scrollable container
- Auto-updates based on filter selections

Video Card Content

Each card must display:

- Video thumbnail image
- Video title

- Artist name
- Release year

Note: Genre is used for filtering only and should not be displayed on the card.

Data Source

API Endpoint

<https://raw.githubusercontent.com/XiteTV/frontend-coding-exercise/main/data/dataset.json>

Data Structure

The JSON contains:

- `genres` : Array of genre objects with `id` and `name`
- `videos` : Array of video objects linked to genres via `genre_id`

Important

- **DO NOT** copy the JSON file into your project
- Fetch data from the URL as if calling a real backend API
- Implement proper loading indicators during data fetch
- Implement error handling for failed requests

Technical Requirements

Must Have

-  **React** - Must be implemented using React
-  **TypeScript** - Strong advantage (highly recommended)
-  **Responsive Design** - Follow the provided design as closely as possible ("pixel perfect")
-  **Custom Components** - You must implement the video card list and individual video card components yourself

-  **External Libraries** - You may use third-party libraries for the dropdown components
-  **Loading States** - Show loading indicators while fetching data
-  **Error Handling** - Gracefully handle API errors with user-friendly messages
-  **Clean Code** - Code should be readable, maintainable, and well-organized

Code Quality Expectations

- Clear component structure and organization
- Appropriate separation of concerns
- Meaningful variable and function names
- Efficient rendering and state management
- Be prepared to explain your architectural decisions

Design Reference

Please refer to the provided design mockup and implement the UI as closely as possible to the design specifications. Pay attention to:

- Spacing and padding
- Typography and font sizes
- Colors and visual hierarchy
- Component alignment
- Responsive behavior

Deliverables

1. Source Code

- Well-organized React application
- Clear folder structure
- [README.md](#) with setup instructions

2. Documentation (in README)

- How to install dependencies
- How to run the application
- Brief explanation of your component architecture
- Any assumptions or decisions you made

3. Bonus Points

- TypeScript implementation
- Unit tests for key components
- Dynamic filter updates based on search results
- Smooth animations and transitions
- Accessibility considerations
- Performance optimizations

Getting Started

1. Set up a new React project (you may use Create React App, Vite, or your preferred tooling)
2. Fetch the video data from the provided API endpoint
3. Implement the filter components in the header
4. Create the video card component and grid layout
5. Wire up the filtering logic
6. Polish the UI to match the design
7. Test your application thoroughly

Resources

- You may use any online resources, documentation, or references you find appropriate
- Feel free to reach out if you have any questions or need clarification
- AI assistance tools are allowed (we use them in real work too!)

Submission

Please submit:

- Link to your Git repository (GitHub, GitLab, etc.)
- Ensure the repository includes clear README instructions
- Include any notes about your implementation

Evaluation Criteria

Your submission will be evaluated based on:

1. **Functionality** (40%)

- All features work as specified
- Filters work correctly individually and in combination
- Proper loading and error states

2. **Code Quality** (30%)

- Clean, readable, and maintainable code
- Good component structure and organization
- Appropriate use of React patterns and hooks

3. **UI/UX** (20%)

- Visual match to the design
- Responsive layout
- User-friendly interactions

4. **Best Practices** (10%)

- Error handling
- Performance considerations
- Accessibility basics

Time Expectation

Estimated Time: 4-6 hours

This is a guideline, not a strict limit. We understand candidates have different schedules and approaches. Focus on quality over speed.

Questions?

If you have any questions or need clarification on any requirements, please don't hesitate to reach out. We're here to help!

Good luck, and we look forward to seeing your work! 🚀